

# Tuple, Liste, Dizionari Python

Advanced Computer Programming

Prof. Raffaele Della Corte

# Sommario

- Tuple
- Liste
- Dizionari

## Riferimenti

- Tony Gaddis. **Introduzione a Python**. 5° ed. – Pearson, 2021
- Paul J. Deitel, Harvey M. Deitel. **Introduzione a Python. Per l'informatica e la data science.** Pearson, 2021
- **Python: How to Think Like a Computer Scientist interactive edition** https://runestone.academy/runestone/books/published/thinkcspy/index.html
- Allen Downey. **Think Python** - https://greenteapress.com/thinkpython2/thinkpython2.pdf

# Tuple

- E’ una **sequenza ordinata di elementi** con (eventualmente) tipi diversi
- Non è possibile cambiare i valori degli elementi, **immutable**
- Rappresentate con parentesi

```
t = ()
t = (2,"mit",3)
t[0]                    → ritorna 2
(2,"mit",3) + (5,6)     → ritorna (2,"mit",3,5,6)
t[1:2]    → slice tuple, ritorna ("mit",)
t[1:3]    → slice tuple, ritorna ("mit",3)
len(t)    → ritorna 3
t[1] = 4  → gives error, can’t modify object
```

empty tuple

extra comma means a tuple with one element

# Tuple

- Di solito usate per eseguire uno **swap** tra valori di variabili

```
x = y
y = x
```



```
temp = x
x = y
y = temp
```



```
(x, y) = (y, x)
```



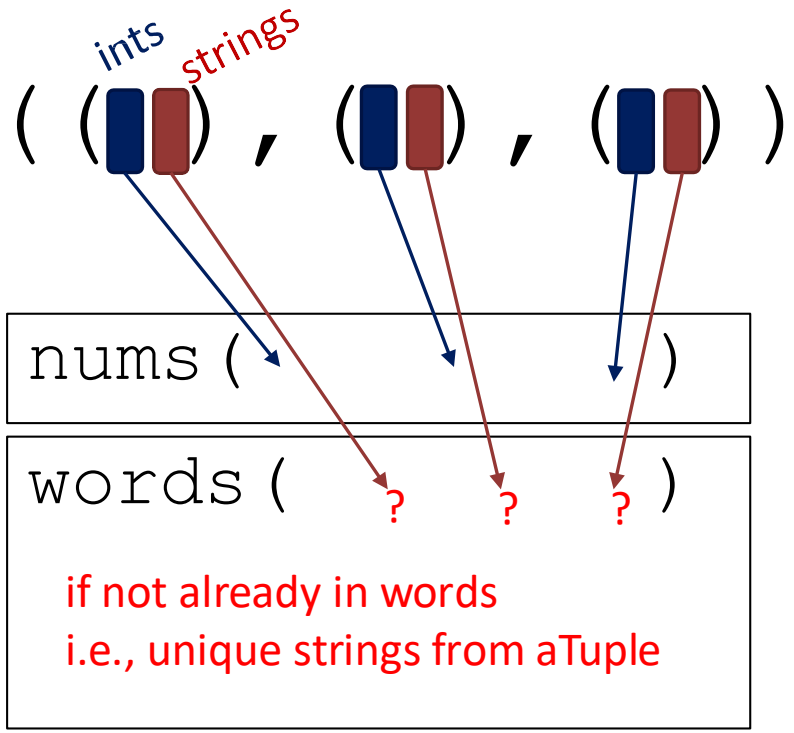
- Utilizzate per **ritornare più di un valore** da una funzione

```
def quotient_and_remainder(x, y):
    q = x // y
    r = x % y
    return (q, r)
(quot, rem) = quotient_and_remainder(4,5)
```

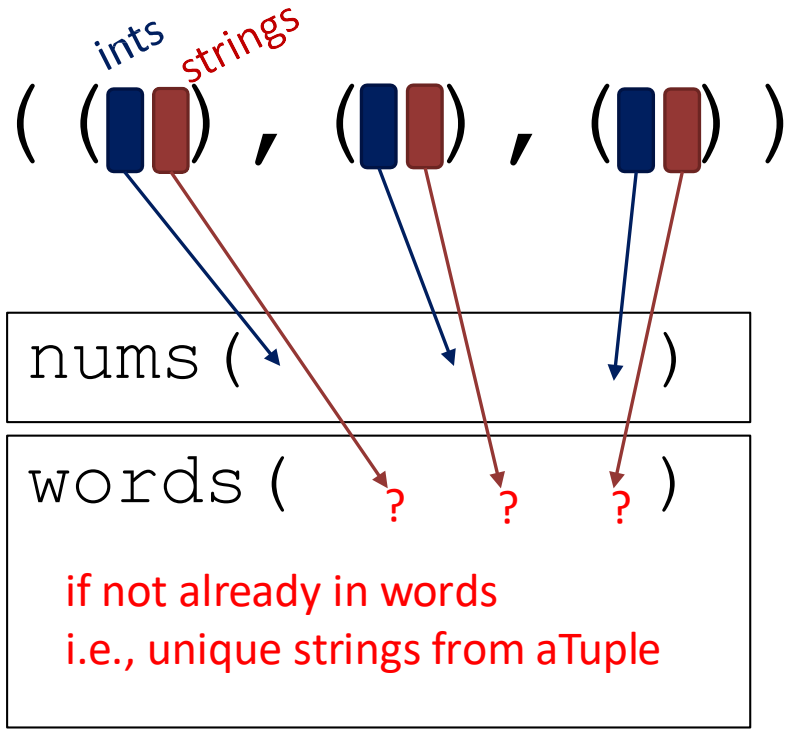
integer division

# Manipolazione delle tuple

- **aTuple** è tupla di tuple di tipo (int, string)
- Sviluppare una funzione ***get_data(aTuple)*** che:
  - Estrae tutti gli interi da *aTuple* e li setta come elementi in una nuova tupla.
  - Estrae tutte le stringhe uniche da *aTuple* e le setta come elementi in una nuova tupla.
  - Ritorna la tuple del minimo e del massimo nella tupla di interi, e il numero di stringhe unique.



# Manipolazione delle tuple




- E' possibile **iterare** sulle tuple

empty tuple

singleton tuple

```
def get_data(aTuple):
    nums = ()
    words = ()
    for t in aTuple:
        nums = nums + (t[0],)
        if t[1] not in words:
            words = words + (t[1],)
    min_n = min(nums)
    max_n = max(nums)
    unique_words = len(words)
    return (min_n, max_n, unique_words)
```

# Liste

- Una **sequenza ordinata** di informazioni, accessibile tramite un *indice*
- Una lista è denotata da **parentesi quadre**, `[ ]`
- Una lista contiene **elementi**
  - **di solito omogenei** (i.e., tutti interi) …

    ma può contenere valori di tipi misti (**non comune**)
- Gli elementi di una lista possono cambiare, quindi una lista è **mutabile**

# Liste: indici e ordinamento



```
a_list = []
```
empty list

```
L = [2, 'a', 4, [1,2]]
```

`len(L)` → ritorna 4

`L[0]` → ritorna 2

`L[2]+1` → ritorna 5

`L[3]` → ritorna `[1,2]`, un'altra lista!

`L[4]` → ritorna un errore

```
i = 2
```

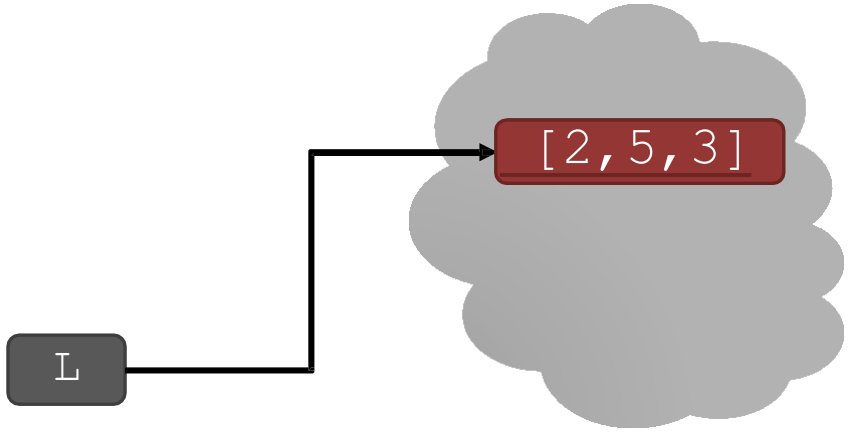
`L[i-1]` → ritorna 'a' perchè `L[1]=='a'`

# Liste: cambiare gli elementi

- Le liste sono **mutabili**
- E' possibile fare l'assegnazione di un elemento ad uno specifico indice cambiando il suo valore

```
L = [2, 1, 3]
L[1] = 5
```

- `L` sarà `[2, 5, 3]`



# Liste: iterare sugli elementi

- **Obiettivo**: elaborare la **somma degli elementi** di una lista
- Il pattern più comune è iterare sugli elementi della lista e calcolare la somma parziale

```
total = 0
for i in range(len(L)):
    total += L[i]
print (total)
```

```
total = 0
for i in L:
    total += i
print (total)
```

like strings, can iterate over list elements directly

- **N.B.:**
  - Gli elementi della lista partono da `0` fino a `len(L)-1`
  - `range(n)` va da `0` fino a `n-1`

# Operazioni su liste: add

- **aggiungere** elementi alla fine della list con `L.append(element)`
- Dopo la chiamata ad append, la lista **muterà**

```
L = [2,1,3]
L.append(5)      → L is now [2,1,3,5]
```



- **L'operatore punto**
  - Le liste sono **oggetti Python**, **tutto in Python è un oggetto**
  - Gli oggetti hanno *dati*
  - Gli oggetti hanno *metodi* e *funzioni*
  - L'accesso a tali informazioni avviene tramite (come in Java/C++) **`object_name.do_something()`**

# Operazioni su liste: extend

- Per combinare più liste insieme si usa il **concatenamento**
- Utilizzando l'operatore **+** otteniamo una nuova lista che è la **concatenazione** delle liste operando
- E' possibile **mutare** una lista con `L.extend(some_list)`

```
L1 = [2,1,3]
L2 = [4,5,6]
L3 = L1 + L2
```

→ `L3` è `[2,1,3,4,5,6]`
`L1, L2` non cambiano

```
L1.extend([0,6])
```

→ `L1` viene mutata in `[2,1,3,0,6]`

# Operazioni su liste: remove

- E' possibile cancellare un elemento ad uno **specifico indice index** con `del(L[index])`
- Per rimuovere un elemento alla **fine della lista** è possibile usare `L.pop()`, che ritorna l'elemento rimosso
- Per rimuovere uno **specifico elemento** è possibile usare `L.remove(element)`
  - Cerca l'elemento e lo rimuove
  - Se l'elemento è presente più volte, rimuove la prima occorrenza
  - Se l'elemento non è nella lista, ritorna un errore

all these operations mutate the list

```
L = [2,1,3,6,3,7,0] # eseguire in ordine
L.remove(2) → muta L = [1,3,6,3,7,0]
L.remove(3) → muta L = [1,6,3,7,0]
del(L[1])   → muta L = [1,3,7,0]
L.pop()     → ritorna 0 e muta L = [1,3,7]
```

# Conversione di liste

- **`L = list(s)`** converte una **stringa in lista** e ritorna una lista che include ogni carattere da `s` in un elemento in `L`
- **`s.split()`** effettua lo **split di una stringa su un carattere specificato** come parametro, utilizzando lo spazio se la chiamata è senza parametri
- **`''.join(L)`** converte una **lista di caratteri in una stringa**, dando un parametro tra singolo apice per aggiungere un carattere tra ogni elemento

```
s = "I<3 cs"          → s è una stringa
list(s)               → ritorna ['I','<','3',' ','c','s']
s.split('<')          → ritorna ['I', '3 cs']
L = ['a','b','c']     → L è una lista
''.join(L)            → ritorna "abc"
'_'.join(L)           → ritorna "a_b_c"
```

# Altre operazioni su Liste

- **sort()** e **sorted()**
- **reverse()**
- e molte altre…
  - https://docs.python.org/3/tutorial/datastructures.html

```
L=[9,6,0,3]
```

`sorted(L)` → Ritorna una lista ordinata, senza **senza mutare** `L`

`L.sort()` → **muta** `L=[0,3,6,9]`

`L.reverse()` → **muta** `L=[9,6,3,0]`

# Liste e oggetti in memoria

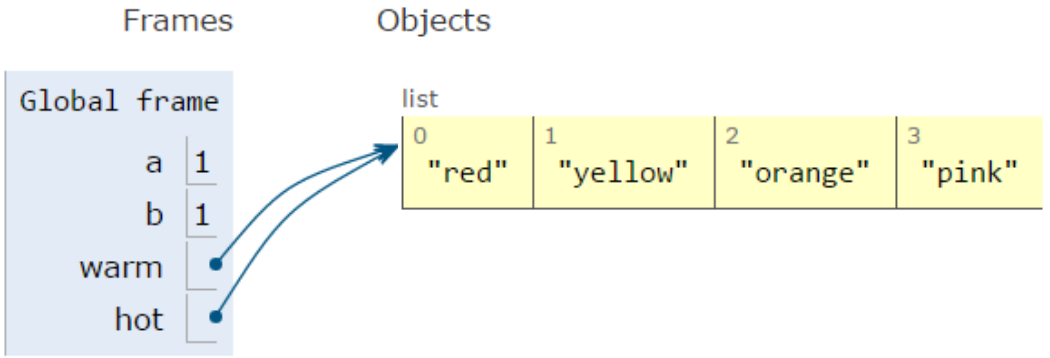


- La lista è **mutabile**
- Si comportano differentemente dai tipi immutabili (stringhe, tuple)
- La lista è un oggetto in memoria
- Il nome di variabile punta ad un oggetto
- Ogni variabile che punta a tale oggetto è impattata da ogni modifica
- La cosa da tenere a mente quando si lavora con le liste è che potremmo avere degli **effetti collaterali!**

# Gli alias

```
1  a = 1
2  b = a
3  print(a)
4  print(b)
5
6  warm = ['red', 'yellow', 'orange']
7  hot = warm
8  hot.append('pink')
9  print(hot)
10 print(warm)
```

```
1
1
['red', 'yellow', 'orange', 'pink']
['red', 'yellow', 'orange', 'pink']
```




- **`hot`** è un **alias** per **`warm`**
  - Il cambiamento di una variabile impatta sul cambiamento dell'altra!
- L'uso di `append()` ha degli effetti collaterali!

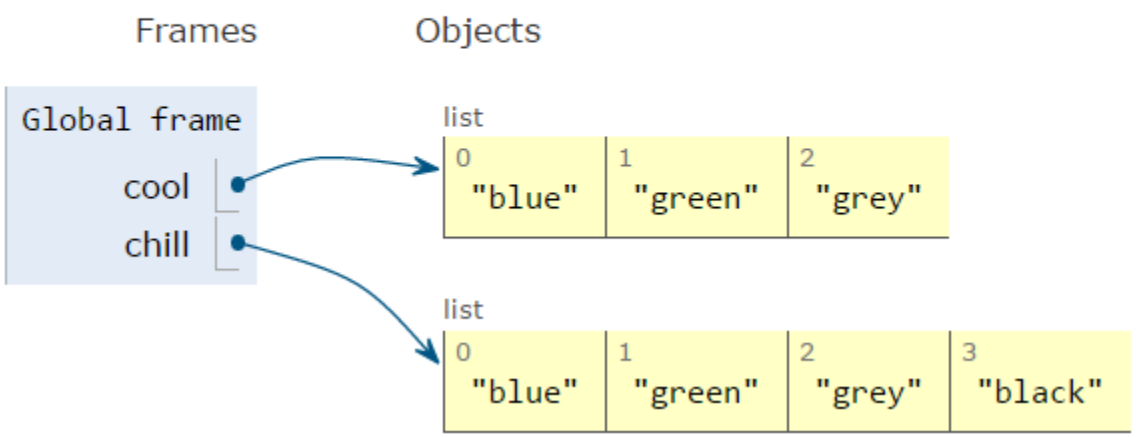
# Clonare una lista

- Clonare una lista significa creare una nuova lista e **copiare ogni elemento** usando l'**operatore [:]**

```
chill = cool[:]
```

```
1  cool = ['blue', 'green', 'grey']
2  chill = cool[:]
3  chill.append('black')
4  print(chill)
5  print(cool)
```

```
['blue', 'green', 'grey', 'black']
['blue', 'green', 'grey']
```

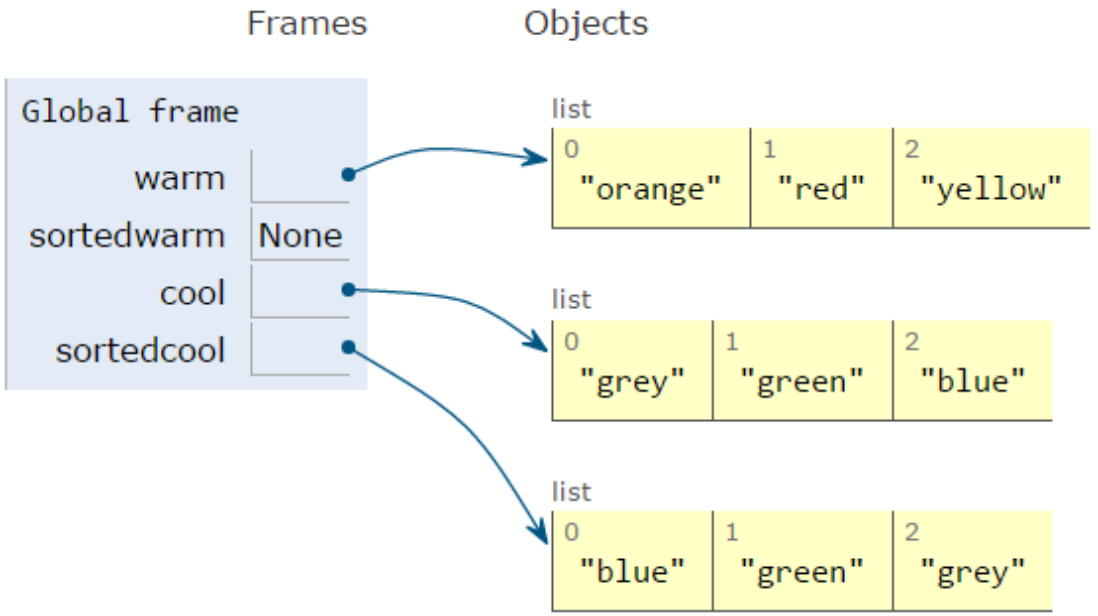


# Ordinamento di una lista

- La chiamata a `sort()` **muta** la lista, ritornando `None`
- Invece, la chiamata a `sorted()` **non muta** la lista, è necessario assegnare il valore risultato ad una variable

```
['orange', 'red', 'yellow']
None
['grey', 'green', 'blue']
['blue', 'green', 'grey']
```

```
1  warm = ['red', 'yellow', 'orange']
2  sortedwarm = warm.sort()
3  print(warm)
4  print(sortedwarm)
5
6  cool = ['grey', 'green', 'blue']
7  sortedcool = sorted(cool)
8  print(cool)
9  print(sortedcool)
```

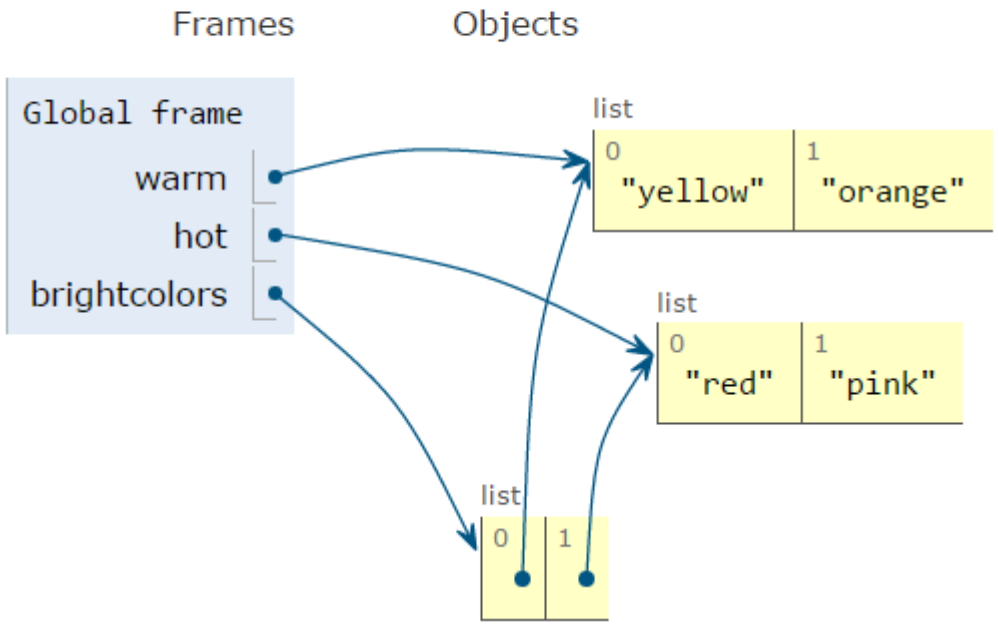


# Liste di liste

- Python permette di avere **liste innestate**
- E' possibile incorrere in effetti collaterali dopo la modifica delle liste innestate

```
[['yellow', 'orange'], ['red']]
['red', 'pink']
[['yellow', 'orange'], ['red', 'pink']]
```

```
1 warm = ['yellow', 'orange']
2 hot = ['red']
3 brightcolors = [warm]
4 brightcolors.append(hot)
5 print(brightcolors)
6 hot.append('pink')
7 print(hot)
8 print(brightcolors)
```



# Mutazione e iterazione su liste

- **Evitare** la modifica di una lista mentre iteriamo sugli elementi

```
def remove_dups(L1, L2):
    for e in L1:
        if e in L2:
            L1.remove(e)
```



```
def remove_dups(L1, L2):
    L1_copy = L1[:]
    for e in L1_copy:
        if e in L2:
            L1.remove(e)
```



clone list first, note that `L1_copy = L1` does NOT clone

```
L1 = [1, 2, 3, 4]
L2 = [1, 2, 5, 6]
remove_dups(L1, L2)
```

- Se usiamo `remove_dups` `L1` è `[2,3,4]` non `[3,4]`, perché?
  - Python utilizza un contatore interno per tenere traccia dell'indice nel ciclo
  - I cambiamenti (e.g., rimozione) **mutano la lunghezza della lista** ma Python non aggiorna tale contatore!
  - Nel ciclo dell'esempio, non vedremo mai l'elemento con valore 2

# Come memorizzare le informazioni di studenti?



- Finora, possiamo memorizzare tali informazioni mantenendo liste separate

```
names = ['Ana', 'John', 'Denise', 'Katy']
grade = ['B', 'A+', 'A', 'A']
course = [2.00, 6.0001, 20.002, 9.01]
```

- Una **lista separata** per ogni elemento (item)
- Ogni lista deve avere la **stessa lunghezza**
- Le info memorizzate per tutte le liste allo **stesso index** fanno riferimento a info per la stessa persona

# Aggiornare/Ottenere info degli studenti



```
def get_grade(student, name_list, grade_list, course_list):
    i = name_list.index(student)
    grade = grade_list[i]
    course = course_list[i]
    return (course, grade)
```

- **Genero confusione** se ho molte info di cui tener traccia
- Devo mantenere **molte liste** e utilizzarle come argomento
- Devo **sempre indicizzarle** tramite interi

# Il dizionario

- Utili per **indicizzare elementi di interesse direttamente** (non per forza interi)
- Utili per usare **una sola struttura dati**, no liste separate

Lista

| | |
|---|---|
| 0 | Elem 1 |
| 1 | Elem 2 |
| 2 | Elem 3 |
| 3 | Elem 4 |
| … | … |

index element

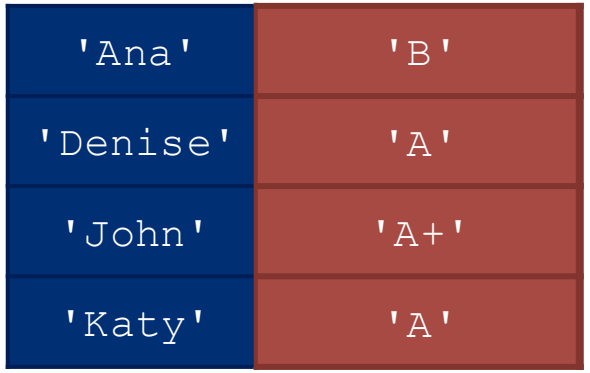
Dizionario

| | |
|---|---|
| Key 1 | Val 1 |
| Key 2 | Val 2 |
| Key 3 | Val 3 |
| Key 4 | Val 4 |
| … | … |

custom index by label element

# Il dizionario in Python

- Memorizza una coppia di valori
  - **key**
  - **value**




empty dictionary

```
my_dict = {}
```

```
grades = {'Ana':'B', 'John':'A+', 'Denise':'A', 'Katy':'A'}
```

key1 val1 key2 val2 key3 val3 key4 val4

# Il dizionario: lookup

- L'operazione di **lookup** è simile ad indicizzare liste
- **Cerca** la chiave (**key**)
- **Ritorna** il **valore** associato con la chiave
- Se la chiave non esiste, ritorna un errore

| | |
|---|---|
| 'Ana' | 'B' |
| 'Denise' | 'A' |
| 'John' | 'A+' |
| 'Katy' | 'A' |

```
grades = {'Ana':'B', 'John':'A+', 'Denise':'A', 'Katy':'A'}
grades['John']      ➔ ritorna 'A+'
grades['Sylvan']    ➔ Ritorna un KeyError
```

# Operazioni su dizionari

| | |
|---|---|
| 'Ana' | 'B' |
| 'Denise' | 'A' |
| 'John' | 'A+' |
| 'Katy' | 'A' |
| 'Sylvan' | 'A' |

```
grades = {'Ana':'B', 'John':'A+', 'Denise':'A', 'Katy':'A'}
```

- **aggiungere** una entry

```
grades['Sylvan'] = 'A'
```

- **test** se la chiave è nel dizionario

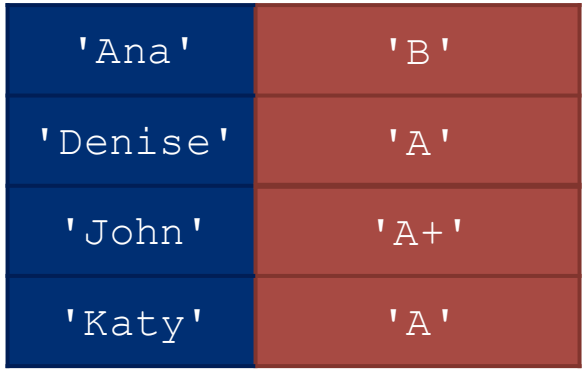
```
'John' in grades
'Daniel' in grades
```

➔ returns `True`
➔ returns `False`

- **cancellare** una entry

```
del(grades['Ana'])
```

# Operazioni su dizionari




```
grades = {'Ana':'B', 'John':'A+', 'Denise':'A', 'Katy':'A'}
```

- Ottenere un **iterabile che sia come una lista di tutte le chiavi**

  `grades.keys()` ➔ ritorna `['Denise','Katy','John','Ana']`

- Ottenere un **iterabile che sia come una lista di tutti i valori**

  `grades.values()` ➔ ritorna `['A', 'A', 'A+', 'B']`

# Chiavi e valori di dizionari

- **Valori**
  - Possono essere di qualunque tipo (**immutabile e mutabile**)
  - Possono essere **duplicati**
  - I valori di un dizionario possono essere liste o addirittura altri dizionari!
- **Chiavi**
  - Devono essere **uniche**
  - Tipo **immutabile** (`int`, `float`, `string`, `tuple`, `bool`)
  - Fare attenzione con tipi `float` come chiavi
- **Non c'è ordinamento** né per le chiavi né per i valori!

```
d = {4:{1:0}, (1,3):"twelve", 'const':[3.14,2.7,8.44]}
```

# Liste vs Dizionari

**Lista**

- Sequenza **ordinata** di elementi
- La ricerca degli elementi è fatta attraverso un indice intero
- Gli indici hanno un **ordinamento**
- Gli indici sono **interi**

**Dizionario**

- **Associa** chiavi con valori
- La ricerca di un elemento è fatta attraverso un altro elemento
- **Nessun ordinamento** è garantito
- Le chiavi possono essere di qualunque tipo **immutabile**

# Esempio dizionario: Analizzatore del testo di una canzone



1) Creare un ***frequency dictionary*** che mappa `str:int`

2) Trovare **la parola più frequente** e quante volte occorre nel testo della canzone
   - Usare una lista nel caso in cui ci siano più parole frequenti
   - Ritornare una tupla `(list,int)` per (words_list, highest_freq)

3) Trovare la **parola che occorre almeno X volte**
   - Lasciare che l'utente specifichi la soglia X
   - Ritornare una lista di tuple, ogni tupla è del tipo `(list, int)` che contiene la lista delle parole ordinata dalla loro frequenza
   - IDEA: Dal dizionario della canzone, trova la parola più frequente, cancellala, ripeti. Funziona perché stiamo mutando il dizionario della canzone.

# Analizzatore del testo di una canzone: creazione dizionario



```
def lyrics_to_frequencies(lyrics):
    myDict = {}
    for word in lyrics:
        if word in myDict:
            myDict[word] += 1
        else:
            myDict[word] = 1
    return myDict
```

can iterate over list

can iterate over keys in dictionary

update value associated with key

# Analizzatore del testo di una canzone: uso del dizionario



```
def most_common_words(freqs):
    values = freqs.values()
    best = max(values)
    words = []
    for k in freqs:
        if freqs[k] == best:
            words.append(k)
    return (words, best)
```

this is an iterable, so can apply built-in function

can iterate over keys in dictionary

# Analizzatore del testo di una canzone: proprietà del dizionario



```
def words_often(freqs, minTimes):
    result = []
    done = False
    while not done and len(freqs)>0:
        temp = most_common_words(freqs)
        if temp[1] >= minTimes:
            result.append(temp)
            for w in temp[0]:
                del(freqs[w])
        else:
            done = True
    return result


print(words_often(beatles, 5))
```

can directly mutate dictionary; makes it easier to iterate